VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI

REPORT ON

BIG DATA ANALYTICS [BCS714D]

Distributed Food and Nutrient Analytics Using MongoDB, Hadoop, Pig and Hive

Track B — Core Themes and Practical Dashboard Extension

Submitted by:

No	Student name	USN	Student signature	Marks awarded	Faculty signature
1	Abhishek R	1JS23CS004			
2	Adithya V	1JS23CS005			
3	Dhanush Singh G	1JS23CS048			

Guided by: Sandhiya B

JSS ACADEMY OF TECHNICAL EDUCATION, Bengaluru – 560060

Department of Computer Science and Engineering

Academic year: 2026–2027

ABSTRACT

This project implements a reproducible food and nutrient analytics workflow using the USDA FoodData Central CSV release dated October 31, 2024. It addresses the three core themes of the BCS714D Big Data Analytics assignment and the Track B practical extension. MongoDB stores the source tables and supports linked document aggregation and index profiling. Hadoop provides HDFS storage and YARN execution of Python Streaming MapReduce. Pig transforms the prepared records, while Hive executes identical queries over Text and gzip RCFile tables. A Flask dashboard presents analytical evidence and controls predefined laptop workflows.

The complete source package was prepared on the project laptop and all 34 CSV tables were imported into MongoDB. The preparation manifest contains 2,063,876 foods, 477 nutrient definitions and 26,848,323 nutrient observations with zero preparation rejects. End-to-end development checks passed for linked subsets of 10,000 and 50,000 foods; the latter contains 1,126,928 linked nutrient observations. In the recorded 10,000-food point-query experiment, an index examined one document and one key instead of scanning 10,000 documents, while returning the same row.

Correctness is established by source fingerprints, input accounting and independent reconciliation of group counts, totals and means. The report distinguishes observed development outcomes from work still pending: full-release analytical benchmarking and execution on three physical college systems. Numerical Hive runtime and storage conclusions await the original selected-run artifacts. The supplied Cloud Report is used only as a presentation reference; all technical content concerns this Big Data Analytics project.

Keywords: FoodData Central, MongoDB, HDFS, YARN, MapReduce, Pig, Hive, RCFile, Flask.

CONTENTS

Abstract....2

CHAPTER 1: INTRODUCTION....5

CHAPTER 2: BACKGROUND AND REQUIREMENT ANALYSIS....7

CHAPTER 3: DATASET AND EXPERIMENTAL SCOPE....9

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE....12

CHAPTER 5: PART 1 — IMPLEMENTATION OF THE CORE THEMES....15

CHAPTER 6: PART 2 — FLASK DASHBOARD AND JOB CONTROLS....19

CHAPTER 7: TESTING, RESULTS AND DISCUSSION....21

CHAPTER 8: DEPLOYMENT AND DEMONSTRATION....25

CHAPTER 9: CONCLUSION AND FUTURE WORK....27

REFERENCES....28

APPENDIX A: REPRESENTATIVE IMPLEMENTATION....29

APPENDIX B: EVIDENCE AND SUBMISSION CHECKLIST....30

LIST OF FIGURES AND TABLES

Figure 3.1 — Core food and nutrient relationships.....10

Figure 4.1 — Data processing and reconciliation workflow.....12

Figure 4.2 — Planned three-system physical deployment.....13

Figure 7.1 — Documents examined in the recorded point-query comparison.....22

Table 2.1 — Assignment requirements and implemented components.....8

Table 3.1 — Recorded full-release preparation and ingestion.....9

Table 3.2 — Available execution scopes.....10

Table 4.1 — Software and key configuration.....14

Table 5.1 — Required MongoDB stages.....15

Table 7.1 — Recorded 10,000-food profiling observations.....21

Table 7.2 — Final measurements to insert from recorded artifacts.....23

Table 7.3 — Current completion and validation status.....23

Table B.1 — Run artifacts and their purpose.....30

CHAPTER 1: INTRODUCTION

1.1 Project background

Food composition data supports nutritional research, product comparison and public information services. USDA FoodData Central distributes food descriptions, nutrient definitions and nutrient observations in separate relational CSV tables. A food can have several nutrient observations, while a nutrient definition supplies its name and measurement unit. The October 31, 2024 package therefore requires linked processing rather than treating each CSV as an unrelated spreadsheet [1, 2].

This project implements a reproducible analytics workflow for the BCS714D Big Data Analytics assignment under Track B. Part 1 covers MongoDB processing, distributed Hadoop MapReduce, and Pig/Hive analytics. Part 2 implements a Flask dashboard for viewing evidence and controlling predefined laptop workflows. The central engineering objective is to obtain consistent analytical answers across the tools while preserving the identity and scope of the input data.

1.2 Problem statement

Develop a system that ingests the assigned FoodData Central release, organizes food and nutrient records, executes the three compulsory analytical themes, checks equivalent results and presents measured evidence through a web interface. The final assignment environment requires a master and at least two workers on separate lab computers. Laptop containers provide a development environment, but do not establish completion of that physical deployment requirement.

The system must distinguish full ingestion from completed full analytics, and development results from final submission benchmarks. An interrupted operation must not be reported as successful. The dashboard must also distinguish saved results from a newly running analysis, so that an earlier success cannot be mistaken for current progress.

1.3 Objectives

The objectives are to preserve all source CSV tables in MongoDB; construct linked food documents and analytical pipelines; compare indexed and collection-scan queries; demonstrate HDFS file operations; execute Python Streaming jobs on YARN; clean records with distributed Pig; compare Hive Text and RCFile tables; and expose traceable results through Flask.

A further objective is reproducibility. Each run should retain source fingerprints, counts, configuration context, logs and result comparisons. Failures should preserve diagnostic evidence rather than silently overwrite existing output. Development subsets must include the selected foods and all linked nutrient observations.

1.4 Scope and deliverables

The primary analytical output is a summary grouped by food type and nutrient identifier: observation count, total recorded amount and mean recorded amount. Protein is one supported nutrient, not the sole subject of the project. The current dashboard summarizes groups; it does not provide a product-level nutrition search for an individual item such as a particular chicken product.

This report describes the implementation and the execution evidence available at preparation time. The complete release was prepared and imported into MongoDB on the project laptop. End-to-end development checks passed for 10,000 and 50,000 selected foods. Full-release analytical benchmarking and execution on three physical lab systems remain pending.

CHAPTER 2: BACKGROUND AND REQUIREMENT ANALYSIS

2.1 Technical background

MongoDB stores document collections and supports aggregation stages for filtering, grouping, projection, array expansion and multiple analytical branches. Indexes improve selected access patterns, but their benefit depends on the filter, data distribution and query plan. The executionStats output provides evidence about examined documents and keys rather than relying only on elapsed time [3].

Hadoop separates distributed storage from resource scheduling. HDFS maintains a file namespace and stores blocks on DataNodes; YARN schedules application containers on NodeManagers. Hadoop Streaming allows Python mapper and reducer programs to participate in MapReduce without implementing the full processing logic in Java [4, 5].

Pig expresses data transformations as a sequence of loads, filters and stores. Hive provides SQL-like analytical queries over files. RCFile uses a row-group and column-oriented organization; its behavior depends on workload, compression, table size and execution overhead. This project uses Pig 0.17.0 and Hive 2.3.9 for compatibility with the assigned legacy technologies [6, 7].

2.2 Existing approach and proposed approach

A spreadsheet or a single script can answer small nutrition queries, but it does not demonstrate the required distributed execution, heterogeneous data management or storage-format benchmarking. Independently written calculations also risk comparing different selected records, filtering rules or units.

The proposed approach uses the same linked records and cleaning semantics across analytical systems, then reconciles every analytical group against an independent calculation. It retains a complete raw import separately from the development subset and derived analytical documents. A dashboard presents measurements and their evidence scope rather than substituting example results.

2.3 Functional requirements

Table 2.1 — Assignment requirements and implemented components

Requirement	Implementation	Evidence retained
Document ingestion	Import every CSV into raw MongoDB collections	Import manifest and collection counts
Aggregation and indexing	Required stages, linked documents and indexes	Pipeline snapshots and executionStats
HDFS file management	mkdir, put, get, cat and rm -r	Command logs and byte-checked probe
Distributed MapReduce	Python mapper/reducer through YARN	Job IDs, output and success checks
Pig transformation	Distributed filtering of prepared TSV	Script and cleaned row counts
Hive comparison	Identical queries on Text and RCFile	Answers, timings and storage metrics
Track B extension	Flask results and laptop job controls	Browser checks, states and logs

2.4 Non-functional requirements

Correctness requires exact observation counts and equivalent groups, with controlled tolerance for floating-point totals and means. Traceability requires matching source hashes and clearly labelled dataset scope. Maintainability requires separation of ingestion, preparation, analysis, orchestration and presentation modules. Robust execution requires new output destinations, explicit failure states and preserved diagnostic files.

The laptop workflow must respect limited memory by switching between MongoDB and Hadoop services. The native lab workflow must use the actual workers and avoid accidentally starting substitute container workers. The website therefore provides a view-only mode for the college master. Local job controls accept predefined actions rather than arbitrary user shell commands.

CHAPTER 3: DATASET AND EXPERIMENTAL SCOPE

3.1 Dataset provenance

The input is the official FoodData Central CSV release dated October 31, 2024 [1]. The team downloaded the full ZIP and reported its size as approximately 456 MB in Ubuntu. File size alone is not the identity check: the preparation and import manifests record a SHA-256 fingerprint, and archive verification checks the ZIP contents. The raw CSV files remain in the original ZIP.

Recorded source SHA-256:

```
85b6268117a8f6b8032d67a06ccd51ed1287a53ebaaeb0b3e262165e2820cb9f
```

Table 3.1 — Recorded full-release preparation and ingestion

Item	Recorded value	Interpretation
CSV tables imported	34	Complete raw table inventory
Food records	2,063,876	Rows in the food table
Nutrient definitions	477	Nutrient names and units
Food-nutrient records	26,848,323	Linked observation source rows
Prepared nutrient rows	26,848,323	Successfully written by preparation
Preparation rejects	0	Invalid references/amounts rejected at preparation

These values come from the user's completed full preparation and import output. Zero preparation rejects does not mean that every amount is nonnegative. Preparation retains valid finite negative amounts; the analytical cleaning rule excludes negative amounts later and accounts for them separately.

3.2 Core relational structure

The food table identifies a food by fdc_id and supplies its type and description. The food_nutrient table associates a food identifier with a nutrient identifier and a recorded amount. The nutrient table supplies the nutrient name and unit. Additional release tables

retain supporting product, sample, method and classification information even when they do not participate in the primary nutrient summary.

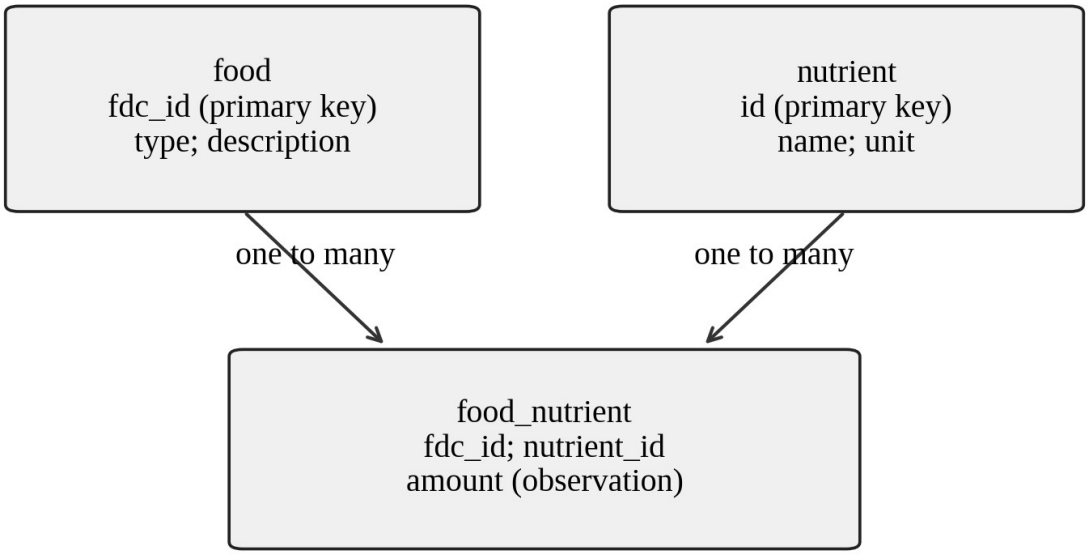


Figure 3.1 — Core food and nutrient relationships

The preparation layer joins these three core tables into a seven-column record containing food ID, food type, category, nutrient ID, nutrient name, unit and amount. This normalized TSV is suitable for line-based Hadoop input. Category values are preserved as nullable text because source values may be identifiers or labels; treating every category as an integer previously caused a preparation failure.

3.3 Development subsets

A development subset selects the first requested foods by source row ordinal and includes all linked nutrient rows for those foods. It also copies the nutrient definitions. This is deterministic selection, not random or representative sampling. The selected food count and nutrient-row count describe different entities and must not be used interchangeably.

Table 3.2 — Available execution scopes

Scope	**Foods / linked observations**	**Recorded outcome**
Full release	2,063,876 / 26,848,323	Preparation and raw MongoDB import completed
10,000-food subset	10,000 / linked rows	MongoDB and distributed development checks passed

Scope	Foods / linked observations	Recorded outcome
50,000-food subset	50,000 / 1,126,928	MongoDB and distributed development checks passed
Synthetic fixtures	Small, deliberately constructed inputs	Implementation correctness checks only

The recorded 10,000-food selection contains 9,994 branded foods and six experimental foods. Its food-type distribution is heavily skewed, so it cannot support general conclusions about the nutritional distribution of the complete release. The 50,000-food subset provides a larger functional test, but does not replace full-scale resource and performance validation.

3.4 Meaning of analytical amounts

For a fixed food-type and nutrient group, the observation count is the number of accepted recorded measurements. Total amount is their sum, and mean amount is that sum divided by the count. Missing observations are not automatically replaced with zero. A stored zero is a valid value and remains in the calculation.

For example, three illustrative protein observations of 2 g, 4 g and 9 g produce a total of 15 g and a mean of 5 g. These illustrative values are not measured USDA project output. The calculation summarizes recorded observations; it is not the nutrition of a single named food, an amount consumed by a person, or a population-weighted dietary statistic. Comparison also requires attention to the source measurement basis; aggregation does not itself harmonize all serving or reporting contexts.

CHAPTER 4: SYSTEM DESIGN AND ARCHITECTURE

4.1 Overall workflow

The architecture separates raw storage, disk-backed preparation, distributed analytical execution and evidence presentation. MongoDB preserves original source strings and supplies document analytics. Python and SQLite prepare linked records without keeping the complete relational input in memory. HDFS stores the prepared files, MapReduce calculates summaries, Pig cleans records, and Hive compares storage/query alternatives. An independent reference scan checks the analytical outputs before the report or dashboard presents completion.

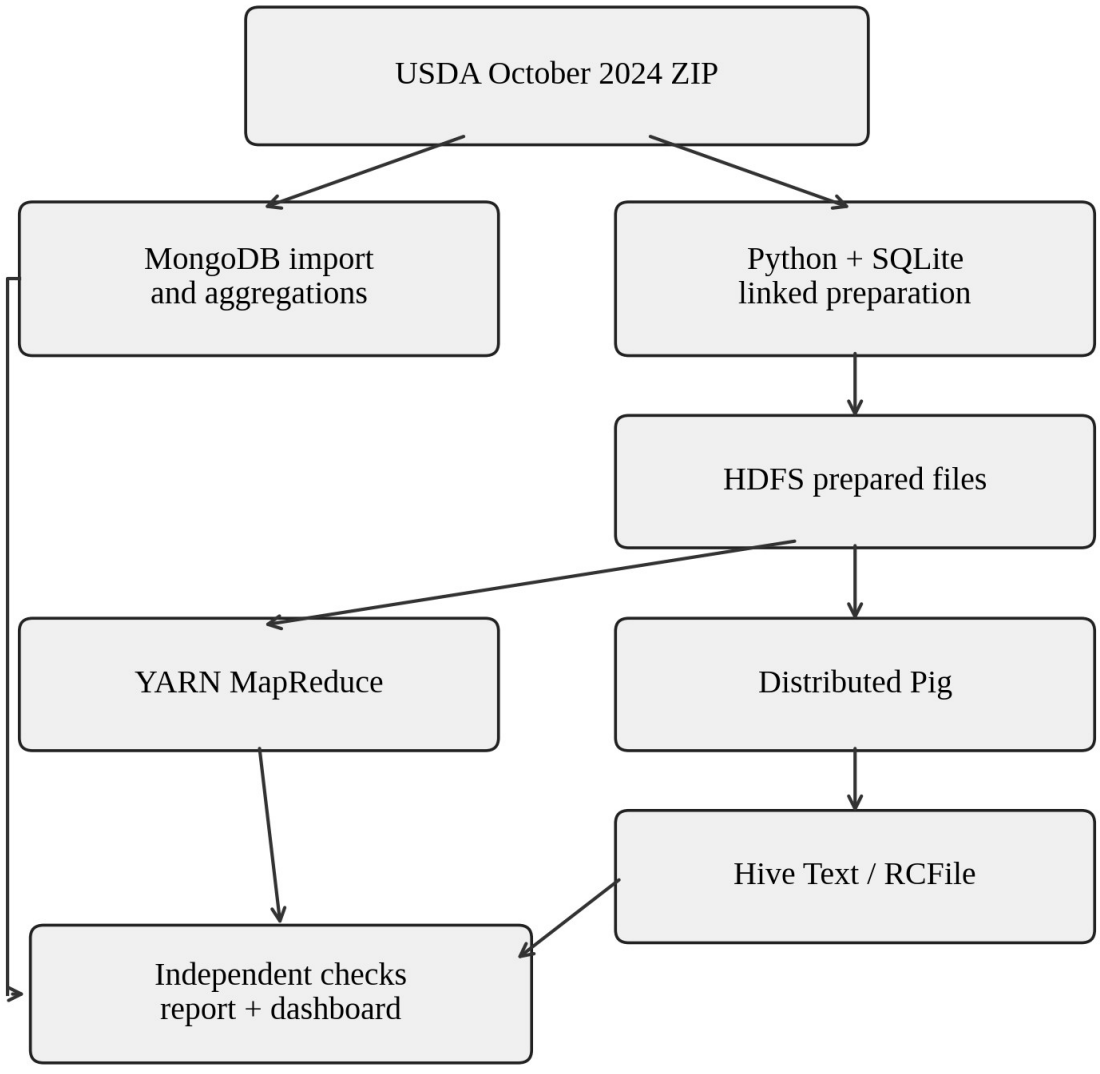


Figure 4.1 — Data processing and reconciliation workflow

The source SHA-256 connects the imported data and prepared artifacts. Development exports additionally identify the selected records and parent release. The reconciliation layer

rejects incompatible source identities rather than accepting similar-looking totals. This prevents accidental comparisons between different subsets, interrupted exports or unrelated saved runs.

4.2 Physical and container deployment

In the required lab deployment, MASTER runs the NameNode, ResourceManager and client-side project commands. WORKER1 and WORKER2 run DataNodes and NodeManagers. MongoDB, HiveServer2 and the single-user metastore are located on MASTER. MongoDB is not sharded across the workers. SQLite is a local preparation file and is not a distributed query engine.

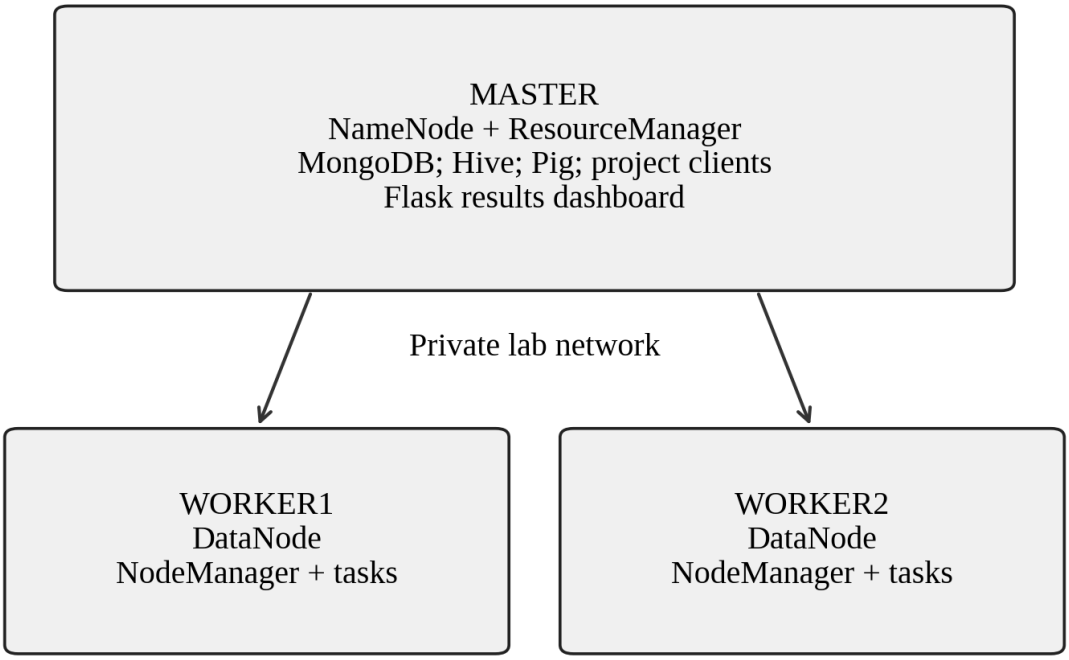


Figure 4.2 — Planned three-system physical deployment

The laptop deployment uses three Docker containers for the Hadoop master and workers. They share one laptop's underlying processor, RAM and physical disk. Registration of two container workers demonstrates a logical cluster, but does not prove three independent physical machines, physical fault isolation or additional aggregate hardware capacity.

4.3 Software and resource configuration

Table 4.1 — Software and key configuration

Component	Version / setting	Role
Operating system	Ubuntu 24.04 / WSL2 for laptop	Execution environment
Java	Java 8	Legacy Hadoop/Pig/Hive compatibility
Hadoop	2.10.2	HDFS, YARN and Streaming
Pig / Hive	0.17.0 / 2.3.9	Transformation and analytical SQL
MongoDB	7.0.16	Raw and derived document collections
Python libraries	PyMongo 4.10.1; Flask 3.1.2	Database access and web interface
HDFS replication	2	Two block replicas where workers permit
YARN per worker	2,560 MiB; 2 vcores	Configured task resources

The project laptop has approximately 15.3 GB installed RAM, with WSL subsequently configured to expose about 9.7 GiB. The development Compose configuration limits each Hadoop container to 4 GiB; these are individual ceilings, not guaranteed simultaneous allocations. The UI stops MongoDB before starting Hadoop and stops the container Hadoop stack before MongoDB analytics to reduce pressure on laptop memory.

4.4 Data and run-state design

Raw collections preserve source-row identity and CSV fields. Import metadata records the release, counts and completion. Derived food documents hold linked nutrient arrays. Prepared files, exported subset metadata and query results are stored separately.

Unique run directories retain job states, logs and atomic progress snapshots. A process lock prevents overlapping UI jobs. States include queued, running, succeeded, failed and interrupted. The analysis page starts at Ready; previous results remain in history.

CHAPTER 5: PART 1 — IMPLEMENTATION OF THE CORE THEMES

5.1 Theme 1: complete document ingestion

The ingestion program iterates over all CSV members of the selected archive and creates a raw_<table> collection for each table. It inserts records in bounded batches, preserving CSV strings and source-row identity. After each table, it compares the inserted count with a database count and updates the import manifest. The manifest becomes complete only after the table inventory has been processed.

The importer refuses a destination database that already contains collections. This protects an existing import from accidental mixing or overwrite. The recorded laptop import completed all 34 tables. For laptop UI analysis, the fixed full source database is fdc_home202410; development subsets use separate generated databases. A native college run creates its own fresh import through the Part 1 driver.

5.2 Linking and indexing

The analytical preparation creates an index on raw_food_nutrient.fdc_id and an index on raw_nutrient.id. A nested lookup associates each food with its nutrient rows and confirms a known nutrient definition. Numeric conversion admits finite amounts and valid nutrient IDs; the derived foods collection contains the linked observations. The derived food count is checked against the input manifest.

An additional index on raw_food.fdc_id supports the profiling query. Index definitions are retained with the result artifacts. Index creation has a cost in time and storage, and the reported point-query benefit must not be generalized to every aggregation or interpreted as evidence of MongoDB sharding.

5.3 Aggregation pipelines

Table 5.1 — Required MongoDB stages

Stage	Project usage	Analytical purpose
$match	Restrict valid types and nonnegative observations	Consistent filtering

Stage	Project usage	Analytical purpose
$group	Group by food type and nutrient	Counts, totals and means
$project	Select or calculate output fields	Compact analytical records
$unwind	Expand the derived nutrient array	One observation per pipeline row
$facet	Food count, type summary and nutrient coverage	Multiple summaries in one pipeline

The food-type pipeline counts foods by data_type. The nutrient pipeline expands nutrient arrays, filters accepted amounts, groups by food type and nutrient ID, and calculates count, sum and average. The overview pipeline uses a facet to return several summaries. The pipeline definitions are saved with the results to show what actually ran.

5.4 Query performance profiling

The same food-ID equality filter is executed through two forced plans: a collection scan using the natural-order hint and an indexed query using idx_fdc_id. The program alternates plan order across five repetitions per plan. It records returned rows, examined documents, examined keys, server execution time and client wall time. PyMongo's explain command with executionStats verbosity is equivalent to the required MongoDB shell profiling operation.

The comparison is a warm-cache development experiment, not a cold-cache benchmark or a distributed MongoDB measurement. Repeated execution may benefit from cached data. A result of 0 milliseconds represents the server timer's resolution and must not be used as a zero denominator to calculate an infinite speedup.

5.5 Theme 2: HDFS file operations

The HDFS demonstration creates dedicated run directories, uploads the original or exported input and prepared files, reads a probe, downloads it and compares the bytes. It deletes only the run's temporary demonstration directory. The raw/prepared data and analytical results remain available for later stages and evidence inspection.

The commands cover hdfs dfs -mkdir, -put, -get, -cat and -rm -r. A successful round trip is stronger than a command screenshot alone: it confirms that the retrieved probe is the same

as the original. The HDFS report, worker registration and safe-mode checks establish readiness before the dataset jobs start.

5.6 Distributed MapReduce algorithm

The mapper reads the seven-column prepared TSV and validates its field count, nutrient ID and finite amount. Negative amounts are skipped and counted through a Hadoop counter. For each retained record, it emits a composite key consisting of food type and nutrient ID, together with the amount and a count of one.

The framework partitions and sorts intermediate records by key. Each reducer processes one key group, adds observation counts and calculates a numerically stable sum using math.fsum. Mean amount is total amount divided by observations. The implementation does not compute an unweighted mean of mapper means, which would be incorrect for groups of unequal size.

Conceptually, for accepted amounts x1 through xn in group g: count(g) = n; total(g) = sum(xi); mean(g) = total(g) / n. The Streaming launcher specifies YARN as the execution framework, distributes mapper.py and reducer.py through the file cache, requests two reducers by default and checks the HDFS _SUCCESS marker.

A successful YARN application and worker registration do not automatically prove useful task placement on both physical workers. The final lab evidence should include task-attempt placement and application identifiers. Container-host task evidence is kept separate from the college-system requirement.

5.7 Theme 3: Pig transformation

Pig loads the prepared TSV using the explicit seven-column schema. The filter requires a food ID, nutrient ID, food type and amount, and excludes negative amounts. It stores the cleaned observations in a dedicated HDFS output location. The project launches Pig in MapReduce mode rather than local mode.

Preparation and cleaning have different responsibilities. Preparation resolves references and rejects malformed or nonfinite amounts with explicit rejection records. Pig applies the analytical filter to the already normalized structured input. This separation avoids using a naive line-based CSV split on quoted source descriptions containing commas, tabs or newlines.

5.8 Hive Text and RCFile storage

The Hive layer creates an external Text table pointing at the Pig output and an external RCFile table with the same columns. The RCFile table is populated from the Text table, with gzip output compression enabled. Both tables therefore represent the same cleaned observations. Dedicated external locations retain the preceding HDFS data.

The first query summarizes every food-type/nutrient/unit group. The second restricts the records to nutrient 1003 with unit g, providing a protein-focused projection of the same analytical design. Local automatic execution is disabled and the execution engine is set to MapReduce. The queries must return equivalent groups, counts, totals and means before performance is compared.

5.9 Benchmark method and reconciliation

For each query/table pair, the benchmark executes one warmup and then the configured number of measured repetitions in a seeded randomized order. With two queries, two formats and five measured repetitions, the full benchmark contains four warmups and twenty measured executions. The short development comparison uses one measured repetition per pair, or eight executions in total.

Elapsed time includes Beeline client startup; it is not pure storage scan time. The program records individual measurements and summarizes measured runs separately from warmups. It obtains logical bytes and replication-adjusted space from Hadoop's ContentSummary interface. Comparing Text with gzip RCFile measures both layout and compression choices, so the result is not a controlled isolation of file format alone.

An independent reference calculation scans prepared records with memory proportional to the number of analytical groups. It checks consistent units for each nutrient ID and uses compensated accumulation. MongoDB, MapReduce and Hive outputs must match exact group membership and counts. Totals and means use relative and absolute tolerances of 1e-9. Source hashes, preparation accounting and Pig output counts must also reconcile.

CHAPTER 6: PART 2 — FLASK DASHBOARD AND JOB CONTROLS

6.1 Purpose and information layout

The practical extension is a Flask web application that explains the three themes and presents analytical evidence. It separates View results from Run analysis. The saved-results view identifies the selected run, dataset scope and source fingerprint; it shows nutrient summaries, index comparisons, Hive timings, storage measurements and requirement-level evidence.

The requirement sections connect each assignment item with its implementation, code snapshot and measured output where available. Missing evidence from an older run is labelled rather than invented. A result table groups foods by type and nutrient; it should not be presented as a list of individual product measurements.

6.2 Laptop execution workflow

The Run analysis view offers common development sizes, a custom size from one to 100,000 foods, and a separate entire-dataset option. The source database is fixed and not editable. Every selected food includes linked nutrient records. A new subset workflow stops the Hadoop containers, starts MongoDB, copies a linked subset, runs MongoDB analytics, exports matching files, switches services and executes the distributed integration checks.

The full-data option requires the original ZIP and verifies the existing full import against its fingerprint and counts. It reuses the raw import rather than duplicating it, but rebuilds derived analysis and prepares new files. This mode is implemented; it has not been demonstrated as a completed full-release laptop run in the available evidence.

6.3 Current progress and saved history

When there is no active job, the analysis view starts at Ready. It does not automatically select an earlier successful job. Previous stages and technical logs are shown only after the user explicitly selects that history entry. The run launched or followed on the current page remains visible when it finishes; a fresh idle visit returns to Ready.

During execution, the dashboard polls every two seconds. It shows the current stage, a plain-language operation description, elapsed time, an actual child-process identifier when available, and the most recent output with its age. Hive progress includes the current query, table format and warmup/measured status. The operation's elapsed time is not an estimate of remaining time or an invented completion percentage.

6.4 Background execution and safety boundaries

The web request launches a detached worker, so closing the browser does not cancel the analysis. A process lock prevents overlapping UI jobs and survives a Flask restart. Job state, current operations and logs are retained under data/ui_jobs. Failed or interrupted workers are not silently treated as successful; diagnostic evidence is retained before a new run is attempted.

Control requests are restricted to local hosts and checked with a request token and origin validation. The server accepts predefined action names and validated sizes rather than arbitrary shell text. These measures support a local teaching application; the dashboard is not deployed as a public multi-user service. Concurrent Terminal jobs remain outside the UI lock and must not be started alongside a laptop UI job.

6.5 Dashboard behavior on three physical systems

The lab guide starts the workflow from MASTER using fdc.part1 with cluster mode client. The dashboard runs with --view-only, which hides laptop Docker controls and rejects control API requests. It reads the native run's saved outputs after completion. While the native job executes, the terminal, cluster.log, run.json and operation.json provide progress evidence.

The latest interface fixes do not change node roles, IP addresses, Hadoop configuration or installed versions. For this guide, project code is cloned on MASTER; workers receive Java/Hadoop and distributed processing files. Updating the dashboard on MASTER is sufficient for the UI-only changes, without reformatting the NameNode or reinitializing the Hive metastore.

CHAPTER 7: TESTING, RESULTS AND DISCUSSION

7.1 Evidence categories

The report separates three kinds of evidence. Recorded terminal output establishes completed operations on the actual USDA laptop data. Cloud fixture runs exercise the implementation with deliberately small, synthetic inputs. Source inspection establishes what the code implements, but does not by itself establish that an unexecuted full release or physical lab deployment has passed.

The latest recorded implementation suite contains 46 passing Python tests, including MongoDB integration. Browser checks cover idle state, history selection, log clearing, new job submission, live process/query display, active-job reconnection, completed-page reload and mobile layout. These results are retained in the repository validation record and are not measurements of USDA full-scale throughput.

7.2 Full preparation and raw ingestion outcome

The complete source package was scanned during preparation and all 34 CSV tables were imported into MongoDB. The reported food and nutrient observation counts agree with the prepared manifest. This demonstrates full ingestion coverage, but the subsequent full MongoDB analytical materialization was interrupted. Therefore, neither a complete full-data analytics result nor a final full-release runtime is claimed.

The interruption also demonstrates why completion flags and unique output locations matter. A partially written file or an active server-side operation is not sufficient evidence of a successful analytical run. The report treats the completed raw import as reusable input and the interrupted analysis as unfinished work.

7.3 Ten-thousand-food index experiment

Table 7.1 — Recorded 10,000-food profiling observations

Metric	Forced collection scan	Indexed query
Returned documents	1 per execution	1 per execution
Documents examined	10,000 per execution	1 per execution
Index keys examined	0	1 per execution

Metric	Forced collection scan	Indexed query
Recorded times, milliseconds	8, 8, 6, 12, 23	1, 0, 0, 0, 0
Median server time	8 milliseconds	0 milliseconds at timer resolution

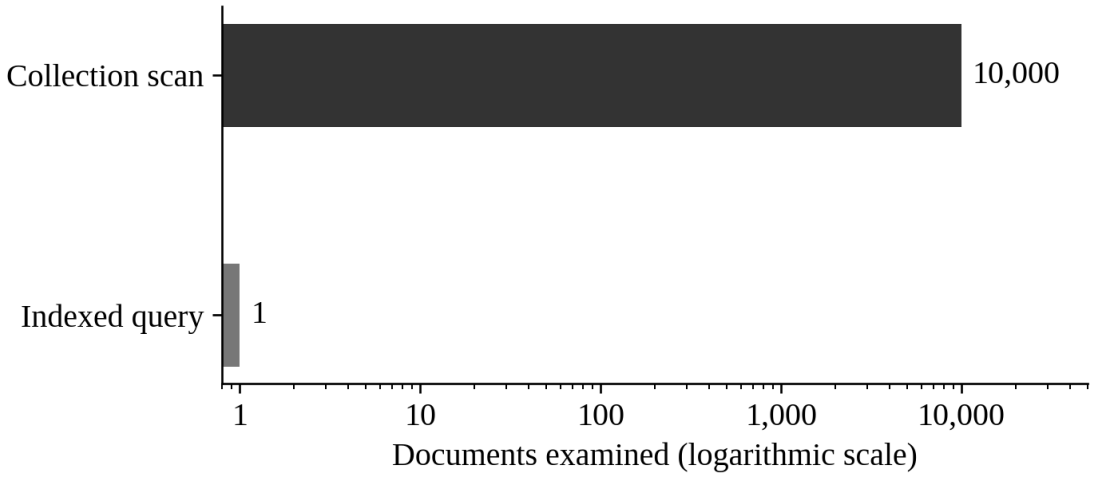


Figure 7.1 — Documents examined in the recorded point-query comparison

The query plans returned the same food while examining very different numbers of records. The indexed query reduced examined documents by 99.99 percent for this particular point lookup. This is a work-reduction observation, not a claim that every query is 10,000 times faster. The zero-millisecond measurements cannot support a finite time-speedup calculation without higher-resolution evidence.

7.4 Fifty-thousand-food distributed outcome

The team completed MongoDB analytics for 50,000 selected foods with 1,126,928 linked nutrient records and 477 nutrient definitions. The program reported ten profiling executions. The exported subset then passed the distributed development checker against matching MongoDB results and independent prepared-data calculations.

The retained laptop evidence path reported by the user is data/realdev_1792244956898668695. Its role is to hold the run's success marker, cluster log, MapReduce output and Hive artifacts. It is a path on the project laptop, not data stored in GitHub. The terminal success message supports a development correctness outcome, but does not supply numerical timing or storage measurements in this report unless those original files are provided.

7.5 Hive and cross-system result interpretation

The available development runs passed output equality checks for the Text and RCFile analytical queries. The project records measured elapsed times and storage sizes, but the user's numerical 50,000-food benchmark files have not been supplied for inclusion here. No runtime or compression ratio is invented. Table 7.2 specifies the measurements that must be transferred from the selected run before a final benchmark conclusion is written.

Table 7.2 — Final measurements to insert from recorded artifacts

Measurement	Required source	Report status
Text query medians	hive/benchmark.json, measured runs only	Await original laptop/lab values
RCFile query medians	Same query and selected records	Await original laptop/lab values
Logical table bytes	Recorded ContentSummary storage	Await original laptop/lab values
Replication-adjusted space	Recorded spaceConsumed values	Await original laptop/lab values
Physical task placement	JobHistory task-attempt records	College execution pending
Full-data reconciliation	reconciliation.json and complete run.json	Full analytics pending

A smaller Text result on a tiny input would not invalidate RCFile; metadata and client overhead may dominate small workloads. Conversely, a compressed RCFile result is not proof of universal superiority. Valid discussion must state the selected input, number of repetitions, cache limitations, compression settings and measurement boundary.

7.6 Functional validation matrix

Table 7.3 — Current completion and validation status

Area	Available validation	Status
Raw release preparation/import	User's complete counts and manifest	Completed on laptop

Area	Available validation	Status
MongoDB subset analytics	10,000 and 50,000-food outputs	Development checks passed
HDFS/YARN/ MapReduce/Pig/Hive	Linked subset integration success	Development checks passed
Dashboard and progress fixes	46-test record and browser checks	Implementation validated
Entire-release analytical execution	Interrupted full analytical run	Pending
Three physical lab computers	Deployment guide prepared	Execution and evidence pending

7.7 Limitations and threats to validity

Deterministic source-order subsets introduce selection bias. Warm-cache query repetition and Beeline startup affect elapsed-time interpretation. One laptop cannot demonstrate physical scaling even when three logical nodes are registered. HDFS-reported virtual capacity must not be mistaken for the Windows host's actual free storage, and replicated bytes must not be confused with logical file length.

Nutrient observations may be sparse, and the current group summaries do not answer every product-level or dietary question. The project does not establish population representativeness, clinical recommendations or uniform serving-basis normalization. Final conclusions about full-data scalability, cluster balancing and storage savings require the planned physical-system measurements.

CHAPTER 8: DEPLOYMENT AND DEMONSTRATION

8.1 Laptop demonstration

On the existing Ubuntu installation, update the checkout and start the dashboard using the project virtual environment. Saved results can be viewed with MongoDB and Hadoop stopped. For a new 50,000-food test, use Run analysis and allow the worker to switch services; do not start duplicate Terminal workflows.

```
cd ~/BDA-Assignment
git pull --ff-only origin main
.venv/bin/python -m fdc.dashboard
```

Choose View results to present an existing evidence folder. Choose Run analysis to process a new selected scope and demonstrate the live stages. The laptop guide and the 50,000-food guide contain the complete command sequence and explain which artifacts are reused. Generated data and results are excluded from the GitHub repository and need a separate backup.

8.2 Three-system deployment procedure

The detailed lab walkthrough installs prerequisites rather than assuming an existing cluster. It assigns one master and two workers, configures matching host mappings and SSH, installs compatible Java/Hadoop versions, prepares HDFS directories and verifies registration before dataset execution. Workers require network access to the actual Hadoop/YARN services, not only SSH.

The master runs the following workflow after the lab guide's preparation and service checks. The run identifier must be new. The dataset ZIP is copied to the master and its CRC and SHA-256 are checked before the full run.

```
.venv/bin/python -m fdc.part1 \
  --archive data/raw/FoodData_Central_csv_2024-10-31.zip \
  --run-id college_full01 --dataset-kind full-release \
  --cluster-mode client --repeats 5
```

All three systems remain awake while the job runs. Never format an existing NameNode or recreate a metastore simply to resolve a website update. Check completion and reconciliation before presenting the dashboard. The physical deployment described here is a prepared procedure, not an experiment already completed on the college machines.

8.3 Evidence to capture for the presentation

Capture the actual master and worker machine inventory, two live DataNodes, two RUNNING NodeManagers and a successful sample job. Retain a Streaming application ID and task-attempt placement on both workers. Show a MongoDB aggregation and its index comparison, the Pig script and cleaned count, and identical Hive answers on Text and RCFile.

Capture screenshots from the user's selected real-data run: the dataset scope, three theme checklists, executionStats table, Hive query/storage comparison and current process panel. Screenshots have not been invented or substituted with synthetic outputs in this report. Insert those screenshots with captions before the final submission.

CHAPTER 9: CONCLUSION AND FUTURE WORK

9.1 Conclusion

The implemented system connects document processing, distributed file storage, Python MapReduce, Pig transformation and Hive benchmarking for the USDA FoodData Central release. The Flask extension adds an understandable interface for recorded evidence and controlled laptop execution. Source fingerprints, explicit scope labels, count checks and independent reconciliation make the results more defensible than isolated tool demonstrations.

The completed full raw import and successful 10,000- and 50,000-food development workflows establish an implemented and tested pipeline at those scopes. The recorded index experiment demonstrates substantially lower examined-document work for a selected indexed lookup. The report does not assert that full-data analytics or the mandated physical lab deployment has been completed.

9.2 Work required before final submission

Execute the full release on the three college systems, record the hardware/network inventory and actual task placement, and retain the successful reconciliation files. Transfer the measured Hive timing and storage results into Chapter 7, include real screenshots, and have the institutional front pages completed and verified. Confirm that the faculty accepts the demonstrated deployment and scope before describing the assignment as fully complete.

9.3 Future enhancements

Future work may add food-description search and individual-product details, a clearer nutrient-name mapping in summary views, additional scientifically appropriate normalization and richer monitoring of resource consumption. A later engineering study could vary split sizes, reducer counts and resource allocations to examine scalability, while keeping that study separate from the chosen dashboard extension.

Production use would require stronger authentication, service isolation, a supported multi-user metastore, backup policy and review of the legacy stack. These are prospective enhancements rather than features claimed by the current teaching implementation.

REFERENCES

[1] USDA Agricultural Research Service, “FoodData Central: Download Datasets,” https://fdc.nal.usda.gov/download-datasets/ . Dataset information for the October 2024 release; reference recorded October 21, 2026.

[2] USDA Agricultural Research Service, “FoodData Central CSV, October 31, 2024,” https://fdc.nal.usda.gov/fdc-datasets/FoodData_Central_csv_2024-10-31.zip . Official source archive URL.

[3] MongoDB, “Explain Results and Aggregation Pipeline,” https://www.mongodb.com/docs/manual/reference/explain-results/ and https://www.mongodb.com/docs/manual/core/aggregation-pipeline/ . Reference for aggregation and executionStats concepts.

[4] Apache Software Foundation, “Hadoop Cluster Setup,” https://apache.github.io/hadoop/hadoop-project-dist/hadoop-common/ClusterSetup.html . Faculty-supplied cluster setup reference.

[5] Apache Software Foundation, “Hadoop Streaming,” https://hadoop.apache.org/docs/r2.10.2/hadoop-streaming/HadoopStreaming.html . Reference for Streaming command execution.

[6] Apache Software Foundation, “Apache Pig 0.17 Documentation,” https://pig.apache.org/docs/r0.17.0/ . Reference for Pig Latin and distributed execution.

[7] Apache Software Foundation, “Apache Hive Language Manual,” https://hive.apache.org/docs/latest/language/languagemanual/ . General Hive language reference; implementation uses Hive 2.3.9.

[8] Pallets Projects, “Flask Documentation,” https://flask.palletsprojects.com/ . Reference for the web application framework.

[9] Dhanush-singh-g, “BDA-Assignment,” https://github.com/Dhanush-singh-g/BDA-Assignment . Implementation, deployment guides and validation record; this report describes repository state through commit 2e9b3f9 and the recorded project discussion.

APPENDIX A: REPRESENTATIVE IMPLEMENTATION

A.1 Pig cleaning script

The following is the implemented distributed transformation. INPUT and OUTPUT are supplied as run parameters. The complete source is pig/clean_nutrients.pig in the repository.

```
raw = LOAD '$INPUT' USING PigStorage('\t') AS (
    fdc_id:long, data_type:chararray,
    food_category_id:chararray, nutrient_id:int,
    nutrient_name:chararray, unit:chararray,
    amount:double
);
clean = FILTER raw BY fdc_id IS NOT NULL
    AND nutrient_id IS NOT NULL
    AND data_type IS NOT NULL
    AND amount IS NOT NULL AND amount >= 0.0;
STORE clean INTO '$OUTPUT' USING PigStorage('\t');
```

A.2 Representative Hive query

The query below is applied to both formats by changing TABLE. It is an extract of the nutrient summary query; the execution wrapper also selects the database and disables automatic local execution.

```
SELECT data_type, nutrient_id, unit,
       COUNT(*) AS records,
       SUM(amount) AS total_amount,
       AVG(amount) AS mean_amount
FROM ${hivevar:TABLE}
WHERE amount >= 0
GROUP BY data_type, nutrient_id, unit
ORDER BY data_type, nutrient_id, unit;
```

A.3 Representative index profiling command

The Python implementation issues the equivalent explain command and retains every response. The indexed and baseline filters select the same food ID; the identifier below is a placeholder to replace with an existing record.

```
db.raw_food.find({fdc_id: '<existing-food-id>'})
  .hint('idx_fdc_id').explain('executionStats')
```

The actual source CSV food identifiers are retained as strings in the raw MongoDB collections. Derived and prepared analytical identifiers are converted as required by their schemas. This distinction explains why the shell example quotes the raw identifier.

APPENDIX B: EVIDENCE AND SUBMISSION CHECKLIST

B.1 Key artifacts

Table B.1 — Run artifacts and their purpose

Artifact	What it establishes
manifest.json	Preparation counts, columns and source hash
MongoDB import metadata	Collection inventory and completion
profile.json	Repeated explain results and dataset scope
activity.json / operation.json	Detailed stages and current/latest operation
cluster.log	Commands and distributed diagnostics
mapreduce.tsv	Reducer counts, totals and means
hive/benchmark.json	Query equality, timings and storage
SUCCESS.json	Completed development integration marker
run.json / reconciliation.json	Full-run stages and independent checks

B.2 Items to complete before submission

Confirm the supplied college, department, team names, USNs, guide and academic year. Obtain any signatures or marks required by the college. Insert the original laptop or college benchmark measurements and authentic screenshots. Complete the full-data run on the required physical systems and retain its application/task evidence. Update the contents page after any edits that change pagination.

The cover signature and marks fields are intentionally blank for the appropriate students and faculty to complete. The formatting uses A4 pages, Times New Roman, bold 14-point headings, 12-point text, justified paragraphs and 1.5-line body spacing. The supplied Cloud Report was used for the cover structure, emblems and department header/footer; its cloud-course content and former guide details were not carried into this report.